SERVICIO NACIONAL DE APRENDIZAJE — SENA

CENTRO DE LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS (CIES)

REGIONAL HUILA

INFORME TÉCNICO DE AJUSTES EN LOS PROCESOS DE DESARROLLO DE SOFTWARE

Evaluación de Comportamiento, Reingeniería de Flujos Operativos y Técnicos Bajo Marcos de Calidad y Metodología Ágil

EVIDENCIA DE APRENDIZAJE:

GA11-220501098-AA3-EV01

Actividad de Aprendizaje: Realiza ajustes en los procesos de desarrollo

PROYECTO FORMATIVO:

YogurASO: Plataforma Web de Comercio Electrónico y Gestión para Yogurt Artesanal (v1.0.0)

APRENDIZ / AUTOR:

Juan Esteban Acosta

PROGRAMA DE FORMACIÓN:

Tecnólogo en Análisis y Desarrollo de Software (ADSO)

Neiva, Huila — Colombia

Octubre de 2026

Ficha Técnica de la Evidencia

Código de Competencia:	220501098 — Evaluar requisitos de la solución de software de acuerdo con las normas y estándares de calidad.
Actividad de Aprendizaje:	AA3 — Evaluar el comportamiento llevado a cabo para realizar los ajustes en el proceso de desarrollo de software de acuerdo con la metodología seleccionada.
Resultado de Aprendizaje (RAP):	220501098-03 — Realizar los ajustes requeridos en los procesos del desarrollo de software de acuerdo con los resultados obtenidos en las pruebas.
Componente Formativo Analizado:	“Aplicación de pruebas de software” — Servicio Nacional de Aprendizaje SENA.
Sistema Evaluado:	YogurASO (Frontend Web, API REST en Node.js/Express, Base de Datos PostgreSQL/Supabase).

1. Introducción y Justificación

El desarrollo de software moderno no es un proceso estático ni lineal; por el contrario, requiere de mecanismos de retroalimentación sistemáticos que permitan identificar desvíos, ineficiencias y cuellos de botella para efectuar mejoras oportunas. En el marco del proyecto formativo YogurASO — una solución web para la comercialización artesanal y gestión de inventarios de yogurt en los municipios de Neiva y Tello (Huila)— se llevó a cabo una exhaustiva fase de verificación y aseguramiento de la calidad.

A partir de los resultados cuantitativos y cualitativos arrojados por las pruebas de software (pruebas de API con Postman, pruebas automatizadas E2E con Playwright y la evaluación bajo la norma ISO/IEC 25010), se identificaron discrepancias operativas y técnicas en el ciclo de vida previo del proyecto. El presente documento formaliza los ajustes efectuados en la secuencia de ejecución de los procesos de desarrollo, detallando la transición metodológica, las modificaciones operativas en la gestión del equipo y las implementaciones técnicas a nivel de arquitectura y código fuente.

2. Reconocimiento de la Metodología Seleccionada

Para el ciclo de desarrollo de YogurASO se adoptó un marco de trabajo ágil basado en Scrum complementado con prácticas de flujo continuo Kanban. Esta selección responde a la necesidad de responder rápidamente a los requerimientos de la microempresa, permitiendo entregas incrementales de valor en periodos cortos denominados Sprints (iteraciones de 2 semanas).

2.1 Evaluación del Comportamiento del Proceso Inicial

Durante las primeras etapas del proyecto se evidenciaron debilidades operativas derivadas de una aplicación inmadura del marco ágil. Específicamente:

- **Enfoque tardío de pruebas (Big Bang Testing):** Las actividades de aseguramiento de calidad (QA) se relegaban al final del Sprint, lo que impedía corregir defectos estructurales sin retrasar la fecha comprometida de entrega.
- **Falta de automatización preventiva:** Los desarrolladores ejecutaban pruebas manuales ad-hoc en sus entornos locales, lo que provocaba que errores de sincronización y regresión pasaran inadvertidos al repositorio compartido.

- **Definición de Hecho (DoD) ambigua:** Las historias de usuario se marcaban como "terminadas" únicamente con la finalización de la codificación, sin requerir cobertura de pruebas automatizadas ni verificación de accesibilidad o rendimiento.

3. Análisis del Componente Formativo “Aplicación de Pruebas de Software”

El estudio riguroso del componente formativo del SENA aporta fundamentos esenciales que sustentan los cambios requeridos en el proyecto:

- **Principio de Detección Temprana (Shift-Left Testing):** El costo y esfuerzo de corregir un defecto se incrementa exponencialmente si se detecta en etapas tardías o en producción. Por ello, las pruebas deben integrarse desde la fase de diseño y concepción de las historias de usuario.
- **Pirámide de Pruebas de Mike Cohn:** La estrategia de pruebas debe fundamentarse en una sólida base de pruebas unitarias rápidas y económicas, una capa intermedia de pruebas de integración/API, y una cúspide reducida pero crítica de pruebas de interfaz de usuario End-to-End (E2E).
- **Independencia y Trazabilidad del Defecto:** Todo hallazgo debe documentarse mediante un formato estandarizado que contenga precondiciones, pasos de reproducción, resultado esperado, resultado obtenido, severidad y prioridad.

4. Propuesta de Cambios en la Secuencia de Ejecución de Procesos

Con el propósito de garantizar que la calidad sea intrínseca al desarrollo y no un control posterior, se rediseñó la secuencia del ciclo de vida del software, estableciendo "compuertas de calidad" (Quality Gates) obligatorias.

4.1 Matriz Comparativa: Proceso Tradicional vs. Proceso Ajustado

Etapa del Proceso	Secuencia Tradicional Previa	Secuencia Ajustada (Shift-Left QA)
1. Planificación del Sprint	Definición básica de alcance técnico sin criterios de aceptación de pruebas.	Diseño colaborativo de criterios de aceptación BDD (Given-When-Then) y escenarios de prueba previos a la codificación.
2. Codificación	Desarrollo aislado de componentes; las pruebas se postergaban hasta tener el módulo completo.	Desarrollo guiado por pruebas (TDD/Component Testing) con linters y pre-commit hooks activos.
3. Integración de Código	Fusión manual de ramas a main sin ejecución automática de pruebas unitarias o de integración.	Integración Continua (CI/CD): Cada Pull Request desencadena validación sintáctica y suite automatizada de Playwright.
4. Verificación y QA	Pruebas manuales masivas al cierre del sprint; saturación y bloqueos de última hora.	Pruebas automatizadas continuas durante todo el sprint; pruebas exploratorias y de usabilidad enfocadas.
5. Cierre y Retrospectiva	Historias cerradas con deuda técnica y correcciones pendientes para futuros sprints.	Revisión estricta de la Definition of Done (DoD), métricas de defectos y lecciones aprendidas documentadas.

5. Aplicación de Cambios a Nivel Operativo

Las modificaciones operativas transformaron la dinámica diaria de trabajo del equipo de desarrollo, fortaleciendo la disciplina ágil:

5.1 Redefinición de la Definition of Ready (DoR) y Definition of Done (DoD)

- **Criterios de Preparación (DoR):** Ninguna historia de usuario pasa al Sprint Backlog si no cuenta con diseño de interfaz validado, casos de uso definidos y especificación técnica de endpoints y datos.
- **Criterios de Terminación (DoD):** Una tarea solo se considera "Terminada" cuando: (a) el código pasa linters sin advertencias, (b) cuenta con pruebas automatizadas aprobadas, (c) fue revisada por pares vía Pull Request, y (d) no rompe la suite E2E en Playwright.

5.2 Gestión y Trazabilidad de Defectos en el Tablero Ágil

Se incorporó una columna de "Auditoría de Calidad" y una política de "Zero Bugs Policy" para incidencias críticas: cualquier defecto que bloquee el flujo de compra o la seguridad del usuario adquiere prioridad inmediata por encima de nuevas funcionalidades.

6. Aplicación de Cambios a Nivel Técnico

A nivel de código fuente y arquitectura, se implementaron soluciones directas a los problemas técnicos identificados durante las pruebas de la solución YogurASO:

6.1 Automatización de Pruebas End-to-End con Playwright

Se configuró una suite automatizada con Playwright para verificar de forma desatendida las rutas críticas del aplicativo: navegación del catálogo, agregado y persistencia en el carrito de compras, cálculo de descuentos y apertura del canal de WhatsApp.

6.2 Corrección de Race Conditions en el Motor de Colorimetría

Se solucionó el problema crítico detectado en la sincronización de temas de sabor entre el carrusel de productos y la portada (Hero): se implementó la bandera isSyncingFromExternal y un canal de eventos maestro-esclavo bidireccional en theme-reactor.js y hero.js, garantizando que el color del fondo y las botellas coincidan siempre con el sabor seleccionado.

6.3 Monitoreo de Salud de Conexión a Base de Datos

Se habilitó la ruta /api/health en el backend de Node.js/Express para verificar en tiempo real el estado de conexión con PostgreSQL y Supabase, permitiendo alertar al frontend antes de que ocurra una falla silenciosa en la experiencia del usuario.

6.4 Optimización de Activos y Compatibilidad Multiplataforma

Se inició la migración de imágenes pesadas hacia el formato moderno WebP y se eliminó el hardcoding de credenciales, consolidando variables dinámicas en .env y config.js para facilitar la portabilidad en cualquier servidor o contenedor.

7. Selección de Buenas Prácticas de Calidad según Marcos de Trabajo

Los ajustes introducidos se alinean con los principales estándares de ingeniería de software reconocidos internacionalmente:

Marco / Estándar	Práctica Seleccionada	Aplicación en YogurASO

ISO/IEC 25010	Evaluación de Calidad de Producto basada en Métricas.	Medición de 16 KPIs en 8 dimensiones (Funcionalidad, Seguridad, Usabilidad, etc.).
ISTQB (International Software Testing Qualifications Board)	Pruebas Tempranas y Agrupación de Defectos.	Diseño de casos de prueba formales previo al desarrollo y trazabilidad de bugs.
Scrum Guide & Agile Alliance	Definición de Terminado rigurosa y Entrega Incremental.	DoD con compuertas de calidad automáticas antes de mezclar código a main.
Clean Architecture & 12-Factor App	Separación de Responsabilidades y Configuración Externa.	División modular frontend/backend/db y centralización en archivos .env.

8. Conclusiones y Evaluación del Impacto de los Ajustes

La implementación de los ajustes en el proceso de desarrollo de YogurASO demostró que la calidad de software no depende únicamente de la habilidad técnica individual, sino de la madurez y disciplina del proceso metodológico:

- **Reducción drástica del retrabajo:** Al incorporar pruebas automatizadas en etapas tempranas, los defectos se corrigen en minutos durante la codificación en lugar de requerir días de depuración al final del sprint.
- **Mayor confiabilidad del producto final:** La resolución de problemas de sincronización de estado y la protección de rutas mediante JWT y bcrypt elevaron la robustez del sistema a un nivel óptimo para su despliegue comercial.
- **Alineación con estándares del sector:** La combinación de Scrum, ISO/IEC 25010 y prácticas ISTQB capacita al equipo de desarrollo en metodologías profesionales de alta demanda en la industria del software.

Juan Esteban Acosta

Aprendiz Tecnólogo en Análisis y Desarrollo de Software (ADSO) — SENA Regional Huila